

What is Flutter?

Flutter is an **open-source UI (User Interface) software development kit (SDK)** created by **Google**. It is used to build **natively compiled applications** for **mobile (Android & iOS), web, desktop (Windows, macOS, Linux), and embedded devices** from a **single codebase**.

Flutter uses the **Dart programming language** and provides a rich set of **pre-designed widgets** that help developers create fast, beautiful, and responsive applications.

Why Should We Use Flutter?

Flutter is popular among students and developers because of the following reasons:

1. Single Codebase

- Write **one code** and run it on **multiple platforms** (Android, iOS, Web, Desktop).
- Saves development time and cost.

2. Fast Development (Hot Reload)

- Flutter supports **Hot Reload**, which allows developers to see changes instantly without restarting the app.
- Very helpful for learning and debugging.

3. High Performance

- Flutter apps are **compiled directly to native ARM code**.
- Performance is close to native Android and iOS apps.

4. Beautiful UI

- Flutter provides **customizable widgets**.
- Smooth animations and modern UI designs are easy to build.

5. Open Source & Backed by Google

- Free to use.
- Regular updates and strong community support.

6. Good Career Scope

- Used by companies like **Google, Alibaba, BMW, eBay**.
- High demand for Flutter developers.

History of Flutter

1. Who Invented Flutter?

- Flutter was developed by **Google**.
- The initial idea came from **Eric Seidel**, a former Google engineer.

2. Why Was Flutter Created?

Google wanted a framework that:

- Can run on multiple platforms
- Gives high performance
- Allows custom UI designs
- Reduces dependency on platform-specific UI components

3. When Was Flutter Introduced?

- 2015**: Flutter was first introduced as an experimental project called **Sky**.
- 2017**: Flutter was officially announced in **Alpha version**.
- December 2018**: Flutter **1.0** was released.

Dart Language (Used in Flutter)

Flutter uses **Dart**, which is also developed by Google.

Features of Dart:

- Easy to learn (similar to C, Java, JavaScript)
- Object-Oriented
- Supports async/await
- Fast execution

Important Flutter Versions Over Time

Flutter Version Timeline

Year	Version	Key Features
2015	Sky	Early experimental version
2017	Alpha	Initial public preview
2018	Flutter 1.0	Stable release for Android & iOS
2019	Flutter 1.12	Web support (beta), desktop preview
2020	Flutter 1.17	Improved performance & UI tools
2021	Flutter 2.0	Stable support for Web & Desktop
2022	Flutter 3.0	macOS & Linux stable support
2023	Flutter 3.x	Performance improvements, Material 3
2024–2025	Flutter 3.x+	AI integration, better tooling & stability

Platforms Supported by Flutter

- Android, iOS, Web, Windows, macOS, Linux, Embedded systems

Where Flutter is Used?

- Mobile Applications
- Web Applications
- Desktop Software
- Startup Apps
- Enterprise Applications

Advantages of Flutter

- One codebase
- Fast UI development
- Near-native performance
- Strong community
- Free & open-source

Disadvantages of Flutter

- App size is larger
- Limited libraries compared to native (but improving)
- Dart language is less popular than JavaScript

Conclusion

Flutter is a powerful framework for students and developers who want to build **modern, fast, and cross-platform applications**. With Google's support and growing demand in the industry, Flutter is an excellent choice for a career in **app development**.

- Flutter is developed by **Google**
- Uses **Dart language**
- First stable version: **Flutter 1.0 (2018)**
- Supports **Android, iOS, Web & Desktop**

Complete Flutter Installation & Setup Guide

System Requirements

- OS: Windows 10+, macOS, or Linux
- RAM: Minimum 8 GB (Recommended)
- Disk Space: At least 10 GB free
- Internet connection

Install Flutter SDK

Step 1: Download Flutter

1. Open a web browser
2. Visit the **official Flutter website**
3. Download Flutter SDK according to your OS (Windows / macOS / Linux)

Step 2: Extract Flutter SDK

- Extract the downloaded ZIP file
- Place it in a fixed location:

Windows

`C:\flutter`

macOS / Linux

`~/development/flutter`

Set Up Environment Variables (PATH)

For Windows

1. Open **This PC** → **Properties**
2. Click **Advanced system settings**
3. Click **Environment Variables**
4. Under **System Variables**, select **Path**
5. Click **Edit** → **New**
6. Add:

`C:\flutter\bin`

7. Click OK and restart the system

Verify Flutter Installation

Open **Command Prompt** / **Terminal** and run:

`flutter doctor`

- This command checks all required dependencies
- Green check means everything is correctly installed

5. Install Android Studio

Step 1: Download & Install

- Download Android Studio from the official website
- Install with **default settings**

Step 2: Required Components

Make sure these are installed:

- Android SDK
- Android SDK Platform-Tools
- Android SDK Build-Tools
- Android Emulator

6. Configure Android SDK Path

1. Open **Android Studio**
2. Go to **Settings** → **Appearance & Behavior** → **System Settings** → **Android SDK**
3. Note SDK path (example):

`C:\Users\YourName\AppData\Local\Android\Sdk`

Flutter usually detects it automatically.

Install Flutter & Dart Plugins

1. Open Android Studio
2. Go to **Settings** → **Plugins**
3. Install:
 - Flutter
 - Dart (auto-installed)
4. Restart Android Studio

Set Up Android Emulator

1. Open Android Studio
2. Go to **Device Manager**
3. Click **Create Device**
4. Select Pixel device
5. Download system image
6. Start the emulator

Create a New Flutter Project (Android Studio)

Steps:

1. Open Android Studio
2. Click **New Flutter Project**
3. Select **Flutter Application**
4. Set Flutter SDK path:

`C:\flutter`

5. Enter details:
 - Project Name: `my_first_flutter_app`
 - Language: Dart
6. Click **Finish**

Flutter creates a default project automatically.

Run Your First Flutter App

Using Emulator

- Start emulator
- Click **Run** button

Using Terminal

```
flutter run
```

Flutter Project Structure

- `lib/` → Main code (`main.dart`)
- `android/` → Android specific files
- `ios/` → iOS specific files
- `pubspec.yaml` → Dependencies & assets

Common Errors & Solutions

Flutter Doctor Issues

- Follow suggested commands
- Install missing SDKs

Emulator Not Detected

- Enable virtualization in BIOS
- Install Intel HAXM / Hypervisor

Flutter Widgets Explained (With Diagrams)

1. What is a Widget in Flutter?

In Flutter, **everything is a widget**.

- Buttons, text, images, layout, padding – all are widgets.
- Widgets describe **how the UI should look**.

Think of widgets like **building blocks** of an app UI.

2. Types of Widgets in Flutter

Flutter widgets are mainly divided into **three categories**:

(A) Stateless Widget

- UI does **not change** once built
- No internal state

Examples: Text, Icon, Row, Column

Diagram (Conceptual):

```
[ StatelessWidget ]
|
UI Fixed
```

(B) Stateful Widget

- UI **can change** during runtime
- Uses `setState()`

Examples: Checkbox, Counter App, Switch

Diagram (Conceptual):

```
[ StatefulWidget ]
|
State Changes
|
UI Updates
```

(C) Layout Widgets

Used to arrange widgets on the screen.

Examples: Row, Column, Container, Stack

Diagram (Row Example):

```
[ Row ]
|||
W1 W2 W3
```

3. Widget Tree Concept

Flutter UI is built as a **Widget Tree**.

Example Diagram:

```
MaterialApp
├── Scaffold
│   ├── AppBar
│   ├── Body
│   │   ├── Center
│   │   └── Text
```

First Flutter App – Counter App

This is the **default Flutter Counter App** created when you start a new project.

1. Full Counter App Code (main.dart)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
.....
```

2. Line-by-Line Explanation

import 'package:flutter/material.dart';

- Imports Material Design widgets

void main()

- Entry point of Flutter app

runApp(MyApp())

- Runs the root widget

StatelessWidget (MyApp)

- App-level widget
- UI does not change

MaterialApp

- Provides theme, navigation, title

StatefulWidget (MyHomePage)

- Used because counter value changes

_counter variable

- Stores counter value

setState()

- Tells Flutter to rebuild UI
- Updates screen when data changes

Scaffold

- Provides basic app layout
- AppBar, Body, FloatingActionButton

FloatingActionButton

- Button to increase counter

Counter App Flow Diagram

User taps + button



`_incrementCounter()`



`setState()`



UI Rebuilds



Counter Value Updated

Learning Summary

- Widgets are building blocks
- Stateless = fixed UI
- Stateful = dynamic UI
- `setState()` updates UI
- Counter App explains Flutter basics clearly